

속도는벡터 · 5/29(금) 기말발표 Q&A 통합본

작성 2026-05-28 18:20 KST · 통합본 (carry: 5/27 31 질문 + 5/28 14:55 갱신본 + 발표 슬라이드 13p + 발표 스크립트 13p + 카톡 5/27-5/28)

본 문서의 위치: 발표자(강재현)는 슬라이드 13p + 스크립트 그대로 발표 진행. 본 문서는 발표 직후 **Q&A 5분 시간**에 청중 (교수님·조교·다른 팀) 질문에 답하기 위한 reference. **조현빈이 실험·method·알고리즘 detail 답변 담당** (카톡 5/26 23:30 합의).

사용법: - 답변 길이 = **15-30초 (구어체)**. 1분 넘어가면 청자 잃음. - 모르면 **"그건 보고서 §X.Y에 자세히 있는데 발표에서는 안 다뤘습니다"** 로 정직 답 - 정보 수치 우선 — 추측 금지 (89.1·5.81·94.9·1.4582·1.4019·1,508 등) - 결합이 평균 latency 0.6% 느린 점·negative finding 등 honest 답

카테고리: - §1 슬라이드 페이지별 청중 즉답 (1~13p 순) - §2 연구 framing - §3 측정 방법론 - §4 결과 해석 - §5 method 별 디테일 - §6 plan 회복 vs latency 비대칭 (★ 가장 까다로움) - §7 통계·정합성 - §8 결합 메커니즘 + negative finding (★ 결정적 질문) - §9 honest limitation - §10 future work - §11 산업 적용 - §12 cheat sheet (외울 핵심 수치) + 발화 톤 룰

§1. 슬라이드 페이지별 청중 즉답 (1p → 13p)

발표 스크립트를 청자 입장에서 읽으면 페이지별로 즉시 발생할 수 있는 의문. 청중이 **"그런데 그게 무슨 뜻이죠?"**라고 끼어들 가능성이 높은 곳들.

슬라이드 1 — 표지

Q1-1. "인덱스 부재 시"라고 하셨는데, 인덱스 있는 경우는 왜 안 봤나요?

인덱스가 있으면 Exquitor가 제안하는 ECQO 메커니즘이 HNSW의 range query로 1~2ms 안에 거의 정확한 카디널리티를 얻습니다. 그건 추정이 아니라 직접 측정이라서 본 연구의 검증 대상이 아니에요. 저희는 추정이 진짜로 필요한 no-index 경로만 봤습니다.

슬라이드 2 — VAQ 정의

Q1-2. SQL의 거리 임계값 0.86은 어떻게 정했나요? 왜 0.86이죠?

그 0.86은 슬라이드 예시 SQL의 임의 숫자예요. 본 측정에서는 selectivity를 0.001·0.01·0.1 세 값으로 두고, 그 selectivity에 맞는 거리 임계값을 데이터 분포에서 역산해서 썼습니다. 즉 "selectivity 0.001을 만드는 거리 threshold"가 측정의 정본 변인입니다.

Q1-3. VAQ가 일반 벡터 검색과 어떻게 다른가요?

일반 벡터 검색은 가까운 K개를 찾는 KNN이에요. VAQ는 SQL 안에 관계형 조건 (조인·필터) 과 벡터 거리 조건이 **동시에** 들어가서 옵티마이저가 두 조건 모두 고려해 plan을 짜야 합니다. 그래서 카디널리티 추정이 plan 전체를 좌우합니다.

슬라이드 3 — 카디널리티·선택도 (영상 슬라이드)

Q1-4. 카디널리티 추정이 틀리면 plan이 구체적으로 어떻게 달라지나요?

대표 예가 join 순서·join 알고리즘 선택입니다. 벡터 조건이 매우 선택적인데 (예: 1만 행) 옵티마이저가 2,000만 행으로 잘못 추정하면 Hash Join을 쓰고 큰 해시 테이블을 만듭니다. 반대로 정확히 1만 행으로 추정되면 Nested Loop이나 Bitmap Index Scan 쪽이 훨씬 빨라요. 영상의 5,677ms vs 977ms 차이가 바로 그 효과입니다.

슬라이드 4 — 기존 시스템 고정 비율

Q1-5. pgvector 33.3%·VBASE 50%·DuckDB-vss 100%는 왜 이렇게 다른가요?

각 엔진이 벡터 조건을 옵티마이저에 추정값으로 넘길 때 쓰는 default heuristic이 달라서예요. pgvector는 일반 부등호 술어에 쓰는 0.333을 그대로 적용하고, VBASE는 더 보수적으로 0.5, DuckDB-vss는 카디널리티 추정 자체를 거의 안 하고 100%로 비활성화한 상태입니다. 모두 데이터·쿼리 분포와 무관한 고정값입니다.

Q1-6. Exqutor의 두 메커니즘 (ECQO·Adaptive Sampling) 중 왜 Adaptive Sampling만 골랐나요?

ECQO는 인덱스가 있을 때 HNSW range query로 정확 카디널리티를 직접 얻는 메커니즘이라 추정 정확도라는 문제 자체가 없습니다. 추정 정확도가 본질적 문제가 되는 건 인덱스 없을 때 — Adaptive Sampling이에요. 그래서 그 단계 하나만 본 거예요.

슬라이드 5 — Adaptive Sampling 흐름

Q1-7. 표본 크기 385는 어디서 나온 숫자인가요?

Bernoulli 표본 크기 결정 공식 $N = z^2 \cdot P \cdot (1-P) / e^2$ 에서 95% 신뢰구간 $z=1.96$, 추정 P 와 허용 오차 e 로 계산하면 385가 나옵니다. Exqutor 논문 §V-B의 표준값이고 저희도 verbatim 그대로 따랐어요.

Q1-8. Adaptive Sampling의 "조정"이 무슨 뜻인가요? N을 어떻게 조정?

Exqutor §V-B의 식 1~6이 momentum 기반으로 N을 동적으로 늘리거나 줄입니다. 누적 Q-error가 크면 N을 늘리고, 작으면 N을 줄여 비용을 아껴요. hyperparameter는 $m=0.9$, $\eta_0=0.1$, $\alpha=50$, $\beta=1.5$, $\gamma=0.99$, $\text{period}=50$ 입니다. 저희는 이 조정 로직은 그대로 두고 N에 대응하는 표본 자체를 어떻게 뽑을지만 바꿨습니다.

Q1-9. 본 연구가 sample size가 아닌 sample selection이라는 게 정확히 무슨 의미?

같은 표본 예산 385개 안에서 어떤 385개를 고를지만 바꿨습니다. 기존은 무작위 베르누이로 균일하게 뽑아요. 저희는 데이터의 분포 정보를 이용해서, 예를 들어 PCA로 차원 축소 후 균등 추출하거나, 공간 곡선 순서로 펼친 뒤 추출하거나, 스트리밍 가중치로 추출하는 13가지 방법을 비교했습니다.

슬라이드 6 — 연구 질문

Q1-10. "데이터 구조를 반영"이 정확히 어떤 정보인가요?

두 가지예요. (1) 벡터 공간에서 데이터가 어디 모여 있는지의 밀도 정보. (2) 데이터의 분산 방향 — PCA의 주성분 같은 거예요. baseline은 이 두 정보를 무시하고 모든 행을 같은 확률로 뽑습니다. 저희는 그 정보를 표본 추출에 반영합니다.

슬라이드 7 — 3가지 추정 방식

Q1-11. 단독 method가 어떤 query에서 크게 틀리나요? 구체 예시는?

다중 벡터 864차원 셀에서 minibatch_partial 클러스터링이 +1,043%~+510% 극단 이상치를 냈어요. 864차원 sparse 분포에서 K-means가 파탄(분산 폭발)하기 때문이에요. 그래서 단독 대체는 1,508 cell 중 35.2%만 baseline을 이깁니다 — 거의 동전 던지기.

Q1-12. 결합이 산술 평균이라는데, 왜 그렇게 단순한 방식인가요?

두 추정량이 독립이고 분산이 비슷할 때 산술 평균이 최소 분산을 줍니다. 가중 평균은 각 추정량의 분산을 알아야 가중치를 정하는데, 그 추정 비용이 추가됩니다. 단순 평균이 가장 robust해요. 후속 연구에서 inverse-variance weighting 같은 변형을 검토할 수 있습니다.

Q1-13. 단독으로 분포 인지만 쓰는 게 왜 안 좋은데, 결합에 넣으면 도움이 되나요?

두 독립 추정의 평균이 분산을 줄이는 통계 효과예요. 단독은 분포 인지 method 자체의 큰 분산 때문에 35%로 약하지만, baseline(베르누이)과 평균내면 분산이 $\sqrt{2}$ 배 줄어서 89%로 뛰어어요. **결합의 효과는 method의 뚝뚝함이 아니라 ensemble 효과입니다.**

슬라이드 8 — 5 데이터셋·4 조작변인

Q1-14. 5 데이터셋 중 측정 대상은 정확히 어떻게 되나요?

DEEP 256차원·SIFT 128차원·SSN++ 144차원 단일 벡터 3개 + DEEP+SIFT, DEEP+WIKI 다중 벡터 2개입니다. YFCC는 본래 framing에 들어 있던 멀티미디어 후보였지만 측정 평면에는 안 들어갔어요. PartSupp_PK는 TPC-H의 partsupp 테이블 primary key — 엔진 측정의 base 테이블입니다.

Q1-15. scale factor와 selectivity 구체 값은요?

scale factor는 1·10·100 세 값 (1GB·10GB·100GB), selectivity는 0.001·0.01·0.1 세 값입니다. 엔진 측정은 가장 안정적인 $sf=10 \cdot sel=0.001$ 평면을 정본으로 썼어요.

Q1-16. 14 method라고 했는데 13 method + baseline 인가요?

네 — baseline (무작위 베르누이) 1개 + 분포 인지 method 13개 = 비교 대상 총 14개입니다. 측정 자체는 16 method였는데 P1 Cluster 계열 3개 (gmm·minibatch_partial·faiss_ivf) 는 다중 벡터에서 파탄해서 권장 제외하고, 강한 13 method만 본 분석에 사용했습니다.

슬라이드 9 — 7 paradigm 13 method

Q1-17. P5 클러스터링이 "1 method"인데 KMeans·GMM 두 개가 그려져 있는데요?

정확한 지적이에요. 슬라이드 시각 단순화 때문에 그려진 거고, 실제 본 분석에 들어간 클러스터링 paradigm method는 권장 제외 후 0개에 가깝습니다 (gmm·minibatch_partial·faiss_ivf 셋 다 다중 벡터에서 파탄). 그래서 13 method 중 "권장 제외된" paradigm으로 봐주시면 돼요.

Q1-18. P6 스트리밍에 ★ 마크가 있는데 무슨 의미인가요?

그 ★는 그 paradigm의 chao_weighted method가 **offline Q-error 측정에서 1위**라는 표시예요. 1,508 cell 전체에서 baseline을 100% (전수) 이겼고, 중앙값 Δ%로는 -6.22%로 가장 강한 개선폭이었습니다.

Q1-19. hyperloglog 같은 해시는 어떻게 분포 인지인가요? 그냥 무작위 해시 아닌가요?

정말 예리한 지적입니다. hyperloglog는 실제로는 분포 무관 무작위 해시예요. 그런데 결합에 넣으면 +4.58% 개선이 나옵니다. 이게 저희 결정적 발견인데, 결합의 효과는 분포 인지 때문이 아니라 **두 독립 추정의 평균이 분산을 줄이는 일반 통계 효과**라는 증거예요. 분포 인지든 무작위 해시든, 독립 추정을 하나 더 넣고 평균만 내도 좋아져요. 보고서 §4.2.3에 honest finding으로 명시했습니다.

Q1-20. 7개 paradigm 분류 기준은요?

표본 선택의 **메커니즘** 차이로 묶었습니다. 공간 곡선·차원 축소·고전 stratification·양자화·클러스터링·스트리밍·해시 일곱 가지. 같은 paradigm 내 method들은 비슷한 강·약점을 보이고, paradigm별로 강한 cell이 달라서 묶어 보는 게 분석에 유리해요.

슬라이드 10 — Q-error 89.1%

Q1-21. 평균 Q-error 1.4582 → 1.4019. 차이 0.06밖에 안 되는데 의미 있나요?

Q-error는 추정값과 실제값의 비율이라 1에 가까울수록 좋아요. 1.46에서 1.40으로 줄었다는 건 추정 오차로는 **13% 감소**입니다. 그리고 평균보다 분포가 더 중요해요. 89%의 cell에서 baseline보다 더 정확하다는 건, baseline이 fragile했던 cell에서 결합이 안정적으로 더 잘 잡는다는 의미예요.

Q1-22. 1,508 cell은 어떻게 세어 나온 숫자죠?

5 데이터셋 × 5 조작변인(A1~A5) × 13 method × 짝지은 셀로, 일부 honest exception 제외 후 1,508 cell이 정보입니다. 같은 cell에서 baseline·단독·결합 세 mode가 동시에 산출되는 matched 측정이예요.

Q1-23. cell이 정확히 무슨 단위인가요?

데이터셋·scale factor·selectivity·K·method가 한 조합으로 고정된 비교 단위예요. 예를 들어 "DEEP·sf=10·sel=0.001·K=20·chao_weighted"가 한 cell이고, 그 안에서 baseline·결합 두 방식의 Q-error를 trial 10회로 비교합니다.

슬라이드 11 — latency·plan 선택

Q1-24. pgvector 기본 5,677ms vs baseline 977.6ms = 5.81x. 이게 왜 이렇게 빠르죠?

pgvector 기본은 33.3% 고정 추정 때문에 partsupp 8천만 행 중 약 2,667만 행으로 카디널리티를 잘못 예측합니다. 그 결과 잘못된 join 방식(Hash Join)을 골라요. 반면 baseline은 베르누이로 385행을 표본해서 실제 카디널리티 (sel=0.001 → 약 8만 행) 에 가까운 추정값을 만들어요. 그게 plan을 더 좋은 쪽 (Nested Loop·Bitmap Scan) 으로 바꿔서 5.81배 빨라집니다.

Q1-25. ★ 그런데 결합이 baseline보다 5.9ms (+0.6%) 느린데, 결합이 더 느린 거 아닌가요?

솔직히 평균 latency만 보면 0.6% 느립니다. 5.9ms 차이는 측정 noise 수준이라 사실상 동등으로 보는 게 맞고, 통계 검정에서도 effect size $|g| < 0.5$ small effect예요. 중요한 건 **plan 선택 안정성**입니다. baseline은 156 plan 중 91개 (58.3%) 만 최적을 잡고, 결합은 148개 (94.9%) 를 잡아요. 같은 평균 속도지만 plan 결정의 robustness가 60→95%로 올라간 게 결합의 진짜 가치입니다.

Q1-26. 156 plan은 어떻게 계산된 숫자인가요?

TPC-H Q3·Q9·Q10·Q12 네 분석 쿼리 × 각 쿼리당 query vector 3종 = 12 cell × 결합 13 method = 156 plan 측정이에요. plan signature 비교는 PostgreSQL EXPLAIN의 노드 타입 pre-order tuple로 합니다.

Q1-27. "최적 plan"의 정의가 정확히 뭔가요? plan signature 일치? 아니면 latency 1등?

실제 실행 시간 기준입니다. 같은 cell에서 1) 정답 카디널리티 (오라클) 주입한 plan, 2) baseline 주입한 plan, 3) 결합 주입한 plan, 4) pgvector 기본 plan을 모두 실행해서 latency 1등 plan을 정답으로 정의하고, 각 inject variant가 그 정답 plan을 만들었는지 봅니다.

Q1-28. latency가 사실상 동등인데 plan 선택률이 58→95%로 바뀐 게 모순 아닌가요?

모순이 아니라 **추정 정확도와 latency의 비대칭 구조**가 본 연구의 중요한 finding이에요. selectivity 0.001의 매우 선택적 cell에서는 baseline도 이미 정답에 가까운 plan을 잡는 경우가 많아 평균 차이가 안 드러나요. 하지만 plan 경계 근처에서 baseline이 흔들리는 걸 결합이 잡아줍니다. 같은 평균 속도라도 워크로드·OS 상태 변화 시 결합이 더 robust하게 정답 plan을 유지해요.

슬라이드 12 — future work

Q1-29. History-aware Adaptive Sampling이 구체적으로 어떤 아이디어인가요?

query vector embedding의 유사도로 과거 쿼리를 클러스터링하고, 비슷한 query 그룹마다 카디널리티 추정값을 신뢰구간과 함께 캐싱합니다. 새 query가 들어오면 가장 가까운 그룹의 캐시값을 1차 추정으로 쓰고, 신뢰구간이 좁으면 sampling 자체를 skip해요. 핵심은 sample budget 자체를 0에 가깝게 줄이는 겁니다. 아직 초안 수준이에요.

Q1-30. 다른 엔진에서 결합 효과가 유지될 가능성은 어떻게 보세요?

pgvector 같은 PostgreSQL 계열 (DuckDB·vss·VBASE·CockroachDB pgvector 호환층) 은 plan-based optimizer라 카디널리티 주입의 효과가 비슷하게 나타날 가능성이 큼니다. 반대로 Milvus·Qdrant 같은 native 벡터 DB는 plan-based optimizer 자체가 약하거나 없어서 효과가 다르게 나올 거예요. 6/11 보고서에서 4 엔진 매트릭스로 검증 예정입니다.

슬라이드 13 — 정리

Q1-31. 한 줄로 핵심 contribution을 말한다면?

"같은 sample budget 안에서 baseline의 무작위 베르누이와 분포 인지 method를 결합하면, 추가 비용 없이 Q-error를 89.1% cell에서 줄이고 plan 선택률을 58.3에서 94.9%로 안정화한다." 한 줄입니다.

§2. 연구 framing

Q2-1. 본 연구는 새 알고리즘 제안인가요, 검증인가요?

검증 연구입니다. 새 알고리즘이 아니라, 학계 정설인 "분포 인지 stratification이 무작위 베르누이보다 정확하다"를 단일 개입 controlled experiment로 검증했어요. 결과는 결합이 baseline을 89% 이기지만 그 메커니즘이 분포 인지 효과가 아니라 두 독립 추정의 평균 효과라는 게 핵심 finding입니다.

Q2-2. Exqutor 논문과 본 연구의 결정적 차이가 뭔가요?

Exqutor는 sample size를 어떻게 동적으로 조절할지에 집중해요. 저희는 같은 sample budget 안에서 표본을 어떻게 뽑을지 한 가지만 바꿨습니다. 추정량 정확도 자체를 보는 연구예요.

Q2-3. 왜 표본 선택 단계 "하나만" 개입했나요? 여러 단계 동시 개입하면 더 좋지 않나요?

단일 개입이어야 효과 분리가 정확해요. 여러 단계를 동시에 바꾸면 89% 우위가 어느 단계에서 왔는지 못 가립니다. ECQO·식 1-6·N=385는 paper verbatim 그대로 두고, 표본 선택 한 단계만 바꿔서 controlled verification 형태로 검증했습니다.

Q2-4. 본 연구를 한 단어로 정의한다면?

"controlled verification" — 알고리즘에서 한 구성 요소를 제거하는 ablation보다 더 엄격해요. baseline·단독·결합 3-way를 동일 조건에서 matched 측정해서 메커니즘까지 분리합니다.

§3. 측정 방법론

Q3-1. 측정 setup 한 줄 요약은?

5 데이터셋 × 5 조작변인 × 13 method × 3 mode = 1,508 cell matched 측정. trial 10회·n_queries 1,000 (Exqutor §V-B verbatim). 엔진 측정은 DEEP sf=10·sel=0.001·TPC-H Q3·Q9·Q10·Q12 4쿼리 × qv 3종 = 12 cell × 13 method = 156 plan. cold cache·sequential 측정.

Q3-2. 왜 엔진은 DEEP sf=10 한 cell만 봤나요? 다른 데이터셋·sf는요?

세 이유예요. (1) sf=10이 서버 256GB RAM에서 cold cache·격리 latency 측정이 가장 안정적, (2) DEEP 256차원이 paper §V-B hyperparameter가 가장 잘 수렴하는 분포, (3) 다른 데이터셋·sf 평면은 offline Q-error 1,508 cell 측정에서 전수 검증되어 결론을 지지합니다. 엔진 측정은 그 결과를 plan recovery·latency로 변환하는 후속 검증이에요.

Q3-3. trial 10회로 충분합니까? 통계 검정력 문제 없나요?

Exqutor 논문 §V-B의 표준 설정이라 동일하게 적용했습니다. 한쪽 꼬리 Wilcoxon 검정 최소 p값이 $2^{(-10)} = 1/1024$ 로 floor에 몰리는 한계가 있어요. 그래서 저희는 **p값이 아니라 $\Delta\%$ ·효과크기 (Cliff's δ ·Hedges' g) ·paired bootstrap 95% CI**로 다측 검정합니다. trial 10 × 1,508 cell × 3 mode = 45,000 짝지는 표본 규모가 결론을 지지해요.

Q3-4. cold cache 격리는 어떻게 보장했나요?

매 trial마다 PostgreSQL bufferpool·OS page cache를 flush하고, 다음 trial 시작 전 5초 sleep으로 디스크 안정화 후 측정했습니다. 또 sequential 측정으로 (4 병렬 contention 회피) latency 격리를 했어요. 4 inject variant × 12 cell × 15 rep × sequential = 약 15시간 측정.

Q3-5. 측정 결과의 재현 가능성은 어떻게 보장하나요?

모든 측정 script와 hyperparameter·seed가 _internal/scripts/ 디렉토리에 commit되어 있고, 정본 raw data는 _internal/cache/rq3/aggregated_v13_full.parquet에 보존됩니다. 같은 서버 환경 + 같은 hyperparameter로 재실행하면 trial 분산 $\pm 2\%$ 안에서 동일 결과를 얻을 수 있어요.

§4. 결과 해석

Q4-1. 1,508 cell 중 89.1% (1,344 cell) 결합 우위 — 의미를 더 풀어주세요.

1,344 cell에서 결합이 baseline보다 Q-error가 낮았다는 의미입니다. 나머지 164 cell (10.9%) 은 baseline이 같거나 약간 더 정확했어요. 89.1%는 cell 단위 better 비율이고, 짝지는 Wilcoxon 검정 + paired bootstrap CI에서 모두 유의미하게 결합이 우세입니다.

Q4-2. 중앙값 $\Delta\%$ -4.38%가 실무적으로 어느 정도 의미 있는 개선인가요?

결합의 Q-error 중앙값이 baseline 대비 약 4.38% 작다는 의미예요. Q-error가 1.5였으면 결합은 약 1.435로 떨어집니다. 실무적으로는 selectivity 0.01·다중 벡터 cell처럼 plan 경계 가까운 cell에서 8 method가 -13% 안팎까지 개선 폭이 커서 실제 plan 결정을 바꿉니다. 그게 §5.3 엔진 단 plan recovery 94.9%로 변환돼요.

Q4-3. plan 회복 94.9% (148/156) 가 무슨 의미인가요?

12 cell × 13 결합 method = 156 측정 중 148개에서 결합이 정답 plan을 회복했다는 의미예요. baseline은 91/156 = 58.3%만 회복해요. 결합이 baseline 대비 +57 plan을 더 회복합니다.

Q4-4. 4-case 매트릭스 (B1·CaseB · TP·FN·FP·TN) 결과는 뭔가요?

156 paired에서 baseline·결합 각각 정답 plan 회복 여부를 2×2 분할합니다. TP 90 (둘 다 정답 57.7%) · FN 1 (B1 정답·결합 망침 0.6%) · FP 58 (B1 오답·결합 회복 37.2%) · TN 7 (둘 다 오답 4.5%). **FP 58 vs FN 1의 비대칭**이 결합의 압도적 dominance를 입증합니다 — 결합이 망친 단 1 cell은 Q3·qid1·sparse_rp의 honest exception입니다.

Q4-5. paradigm 강·약 순위가 어떻게 되나요?

중앙값 Δ% 기준: P3 Streaming -6.22% > P4 DimReduction -4.72% > P2 Spatial -4.66% > P5b Hashing -4.58% > P5 Stratification -4.43% > P6 Quantization -3.52% > P1 Cluster -1.40%. P3 Streaming의 chao_weighted가 1위인 이유는 importance 가중을 동적으로 갱신해 분산이 가장 낮아서고, P1 Cluster가 꼴찌인 이유는 다중 벡터 864차원에서 K-means가 파탄해서예요.

§5. method 별 디테일

Q5-1. 13 method 중 1위는 어떤 거였나요?

chao_weighted (P3 Streaming). offline Q-error 측정에서 중앙값 Δ% -6.22%, 1,508 cell 전수에서 baseline 100% 우위, 엔진에서도 12 cell 중 11 정답 plan 회복. 1-pass 스트리밍이라 다중 벡터 고차원에서 안 파탄해서 가장 robust해요.

Q5-2. chao_weighted가 어떤 알고리즘인가요?

Chao 1982 weighted reservoir sampling 변형이에요. 데이터를 스트림처럼 한 번만 흘리면서 각 행 i 에 가중치 w_i 를 부여하고, 확률 $w_i/\Sigma w$ 로 표본을 선택합니다. 메모리는 표본 크기 K 만 쓰고, 시간은 데이터 크기에 선형이라 효율적이에요. 가중치로는 row의 norm을 썼습니다.

Q5-3. 엔진에서는 어떤 method가 가장 좋았나요?

mhst2 (P6 양자화) 가 엔진 12 cell × 12 정답 plan 회복 = 100%이고 latency Δ% -2.57%로 1위였어요. PCA 2D 환원 후 equi-depth grid bin 방식입니다. **균형 1위는 pca1d** — offline + 엔진 양쪽에서 모두 강했습니다.

Q5-4. method 직관 한 줄 설명을 빠르게 부탁드립니다.

method	paradigm	한 줄 설명
chao_weighted	P3 Streaming	1-pass 가중 reservoir
mhists	P6 양자화	PCA 2D + equi-depth grid
pca1d	P4 차원축소	첫 주성분 1D 정렬 후 균등
hilbert_real	P2 공간곡선	PCA 2D + xy2d Hilbert
skilling_hilbert	P2	다차원 Hilbert (PCA 4D)
zorder_morton	P2	비트 interleave Z-order
ica_fastica	P4	독립 성분 분석
sparse_rp	P4	Li-Hastie-Church 2006
rsvd	P4	랜덤 근사 SVD
cum_sqrtf	P5 stratification	Dalenius-Hodges 1959 누적 $\sqrt{f}$
lavallee_hidirolou	P5	take-all + take-some 분할
rabitq_strat	P6 양자화	1-bit 부호 양자화 후 bucket
hyperloglog	P9 해시	해시 선두 0 개수 (★분포 무관)

Q5-5. P1 Cluster 계열을 왜 권장에서 제외했나요?

다중 벡터 864차원 $sf=10$ cell에서 minibatch_partial이 +1,043%~+510% 극단 이상치를 냈어요. 864차원 sparse 분포에서 K-means가 분산 폭발하기 때문입니다. gmm·faiss_ivf도 일관된 우월성을 못 보여서 paradigm 전체를 권장 제외했습니다. honest limitation으로 §4.7에 명시했어요.

Q5-6. hilbert_real이 사실 진짜 Hilbert 곡선이 아니라면서요?

5/24 적대 검증에서 발견한 부분인데, 본 연구 구현은 PCA 2D 환원 후 Wikipedia xy2d 진짜 2D Hilbert curve algorithm을 적용합니다. 즉 고차원 256·128·144차원 Hilbert 곡선이 아니라 PCA 분산 요약 + 2D 공간 채움 hybrid예요. 측정 수치 자체는 유효하지만 효과의 해석을 "PCA + 공간 채움 hybrid 효과"로 정정했습니다. 진짜 다차원 Hilbert는 skilling_hilbert가 따로 있어요.

§6. plan 회복 vs latency 비대칭 (★ 가장 까다로움)

Q6-1. plan 회복 94.9% vs latency 동등 86.9% — 이 둘이 무슨 아닌가요?

모순이 아니라 추정 정확도와 latency의 비대칭 구조가 본 연구의 가장 중요한 finding이에요. baseline (B1) latency 평균 977.6ms가 정답 (957ms) 과 2.1% 차이로 이미 매우 가까워요. selectivity 0.001의 매우 선택적 cell에서는 "벡터 술어가 매우 선택적"이라는 정보만 전달되면 plan이 충분히 좋은 쪽으로 흘러갑니다. **결합의 가치는 latency 추가 가속이 아니라 plan 선택 robustness 그 자체**예요. 베이스라인은 plan 경계 cell에서 fragile하고 결합은 robust해요.

Q6-2. 결합이 baseline보다 0.6% (5.9ms) 느린데, 결합이 진짜 좋은 건가요?

솔직히 평균 latency만 보면 0.6% 느립니다. 5.9ms는 trial 분산 안의 noise 수준이고, 통계 검정에서 effect size $|g| < 0.5$ small effect로 분류돼요. 168 비교 중 86.9% (146/168) 가 small effect라서 사실상 동등으로 봐주시면 돼요. 진짜 가치는 plan 안정성 — 58.3% → 94.9%로 +36%p 향상.

Q6-3. 같은 평균 속도면 결합의 실용적 가치가 뭐예요? 그냥 baseline 쓰면 안 되나요?

baseline은 plan 경계 cell에서 fragile합니다. 워크로드가 조금만 바뀌거나 OS 상태가 흔들리면 plan flip이 일어나서 latency가 크게 될 수 있어요. 결합은 같은 평균 속도지만 plan을 안정적으로 유지하니까 운영 환경에서 latency 분산이 작아요. p99 latency 같은 tail이 중요한 시스템에서는 그게 결정적입니다.

Q6-4. 5.81x 가속의 본질은 카디널리티 추정의 효과인가요, plan 선택의 효과인가요?

둘 다입니다. 정확한 카디널리티가 plan 선택을 바꾸는 효과예요. pgvector 기본은 33.3% 고정으로 partsupp 8천만 행 중 약 2,667만 행을 잘못 예측해서 Hash Join을 골라요. 정답 카디널리티가 주입되면 매우 선택적이라는 정보가 join 순서까지 모두 더 좋은 쪽으로 흐릅니다. Q3 (5-way join) 가 7배대로 가장 강한 가속, Q9 (2-way) 는 2배대로 작은 이유가 plan space 풍부함의 차이예요.

Q6-5. plan 경계 cell에서 baseline이 fragile하다는 게 무슨 의미인가요?

sel=0.001의 매우 선택적 cell에서도 일부 cell은 카디널리티 추정값이 plan 결정 임계값 (예: PostgreSQL의 work_mem 또는 random_page_cost 기반) 근처에 있어요. baseline의 trial 분산이 그 임계값을 왔다 갔다 만들면 같은 cell에서 trial마다 plan signature가 달라집니다. 결합은 분산이 작아 임계값을 안정적으로 한쪽에 머무르게 해요.

Q6-6. injection_fired = True 100%가 무슨 의미인가요?

패치된 pgvector vector.c의 ExecutorRun 훅이 모든 측정에서 정상적으로 발동되어 카디널리티 주입이 plan 결정에 실제로 영향을 줬다는 sanity check예요. 17 inject variant × 12 cell = 204/204 = 100% fired. 측정한 latency 차이가 injection 실패의 artifact가 아니라 진짜 카디널리티 효과임을 입증합니다.

§7. 통계·정합성

Q7-1. paired bootstrap 95% CI는 어떻게 계산했나요?

B=2,000 resample, cluster paired bootstrap (같은 cell의 baseline·결합을 같이 resample) 으로 95% CI를 산출합니다. 결합 vs baseline의 paired $\Delta\%$ CI는 모든 paradigm에서 0을 포함하지 않아 통계적으로 유의해요.

Q7-2. 효과크기는 어떻게 봤나요?

Cliff's δ (paired ranking effect size) 와 Hedges' g (Cohen's d 의 small-sample 보정) 두 가지로 봅니다. Hedges' g 는 보정 계수 $j = 1 - 3/(4n - 5)$ 를 적용해서 trial 10회의 bias를 줄였어요. 결합 vs baseline은 $|g| > 0.5$ medium effect 이상이 대부분, 결합 vs 정답은 $|g| < 0.5$ small effect로 동등.

Q7-3. multiple comparison 보정은요?

Holm 방법으로 family-wise error rate를 통제합니다. 156 plan $\times$ 14 inject variant = 168 비교에서 $p_{\text{holm}} < 0.05$ 유의가 13개로 7.7%이고, 나머지는 동등으로 분류돼요. 효과크기 분포가 결론을 직접 지지합니다.

Q7-4. q-error 분포가 long-tail인데 평균값을 그냥 쓰는 게 맞나요?

정확한 지적이에요. Q-error는 ratio라서 log scale로 분석하는 게 적합적입니다. 본 연구에서는 (1) 중앙값 $\Delta\%$ (long-tail 둔감) 와 (2) better 비율 (cell 단위, 분포 무관) 을 주 지표로 쓰고, 평균은 보조 지표로만 봅니다. 89.1% better 비율과 중앙값 -4.38% 가 long-tail 이상치에 둔감해요.

§8. 결합 메커니즘 + negative finding (★ 결정적)

Q8-1. 결합 89.1% 우위의 메커니즘이 분포 인지 효과인가요, 평균 효과인가요?

핵심 finding이에요 — 89.1% 우위의 메커니즘은 분포 인지 효과가 아니라 두 독립 추정의 평균이 분산을 줄이는 일반 통계 효과입니다. 메커니즘을 가린 결정적 통제군이 §4.2.3의 CaseC (dual-Bernoulli ensemble) 측정이에요. 분포 인지 자리에 또 다른 독립 베르누이를 두고 평균낸 통제군이 95 cell 100%에서 결합보다 더 정확했어요. 즉 분포 인지 method가 두 독립 추정 평균 위에 양의 효과를 더하지 못합니다.

Q8-2. ★ 그럼 그냥 베르누이를 두 번 뽑아서 평균내도 되는 거 아닌가요?

정말 예리하세요. 저희가 통제군으로 그걸 측정했어요. dual-Bernoulli ensemble (CaseC) — 베르누이 두 번 독립 표본해서 평균낸 거 — 가 95 cell 100%에서 본 연구 결합보다 더 정확했습니다 (paired median $\Delta\%$ -5.98%). 즉 본 연구의 89% 우위는 두 독립 추정 평균의 분산 감소 효과지, 분포 인지의 추가 효과가 아니에요. honest limitation에 명시했습니다.

Q8-3. 그럼 분포 인지 자체는 의미 없는 건가요?

아니에요. 분포 인지 단독으로는 35%로 약하지만, 분산이 다른 두 추정량을 결합할 때 한 쪽을 분포 인지로 두는 게 baseline 분포에 robust한 선택이 됩니다. 본 연구는 단일 개입의 효과만 본 거고, 분포 인지가 가치가 없다고 결론 내린 건 아니에요. 분산 다양화 측면에서 여전히 의미가 있어요.

Q8-4. 89% 우위가 positive result인데 왜 negative finding이라고 하나요?

결합 vs baseline은 positive (89% 우위·plan 회복 94.9%) 입니다. negative는 **결합 메커니즘의 본질** — 분포 인지 효과인지 단순 평균 효과인지를 가렸을 때 분포 인지가 평균 효과 위에 추가 효과를 주지 못한다는 거예요. 즉 학계 정설인 "분포 인지가 무작위보다 정확하다"의 단일 개입 검증에서 negative finding이 나온 거고, 본 연구의 학술적 정직성 그 자체가 contribution입니다.

Q8-5. 그럼 향후 분포 인지를 어떻게 활용하면 되나요?

두 방향이 있습니다. (1) 단순 평균이 아니라 inverse-variance weighting 같은 가중 결합 — 분포 인지 추정량의 분산을 baseline보다 작게 만들 수 있다면 가중 평균에서 양의 효과를 낼 가능성. (2) 다른 sample budget·sample size·N에서는 분포 인지의 추가 효과가 다르게 나타날 가능성. 두 방향 모두 future work입니다.

Q8-6. dual-Bernoulli ensemble을 발표에서 왜 뺐나요?

발표 10분 안에 negative finding의 통제군까지 다루면 청자가 핵심 메시지를 잡기 어려워서, 발표는 핵심 3-way (baseline·단독·결합) 로 한정했어요. 통제군과 negative finding은 보고서 §4.2.3에 풀어 썼습니다. 질문 주시면 자세히 답드립니다.

§9. honest limitation

Q9-1. 본 연구의 한계 세 가지만 정직하게 말해주세요.

(1) **sel=0.001 보수적 평면 한정** — 더 넓은 selectivity 평면 (0.01·0.1) 의 결합 효과는 부분 검증만. (2) **P1 Cluster paradigm 다중 벡터 864차원에서 파탄** — 권장 제외, 다중 벡터 이상치 2건. (3) **trial 10의 p값 floor 1/1024 한계** — 효과크기·paired bootstrap CI로 보완. 보고서 §4.7에 honest limitation 5종을 모두 명시했어요.

Q9-2. 다중 벡터 이상치 2건이 본 결과를 왜곡하지 않나요?

평균 -4.09% → -3.06%로 1%p 끌어올리지만, **better 비율 (89.1%) 과 중앙값 ($\Delta\%$ -4.38%) 은 이상치에 둔감한 robust 통계량**이라 영향 없어요. 본 연구는 평균 대신 better 비율·중앙값을 주 지표로 쓰고, 이상치 2건을 honest limitation으로 정직하게 보고합니다.

Q9-3. method 명칭 정정 8건이 결과에 영향을 주나요?

측정 수치는 영향받지 않습니다. 영향받는 건 method 효과의 해석 설명이에요. 정정 전에는 §4.4 ranking 1·2위 (hilbert_real·skilling_hilbert) 가 "고차원 Hilbert 곡선 효과"로 해석됐는데, 실제 구현이 PCA 환원 + 공간 채움 hybrid라서 그 메커니즘으로 정정했습니다. 결론 (89.1% 우위·5.81× 가속) 은 모두 유효해요.

Q9-4. concat sf=100 일부 미측정이 결론에 영향을 주나요?

다중 벡터 sf=100은 DEEP+SIFT만 측정하고 DEEP+WIKI 등 일부 cell은 서버 시간 한계로 측정 안 했습니다. 1,508 cell 정보는 측정 완료된 cell만 포함하고, 미측정 cell은 §4.7 honest limitation으로 보고합니다. 결론은 측정한 1,508 cell에 한정된다고 명시해요.

§10. future work

Q10-1. 향후 연구 방향 핵심 세 가지는?

(1) 더 넓은 selectivity 평면 (0.01~0.1~0.5) 에서 plan recovery 효과 변환 검증. (2) 다중 벡터 (한 SQL에 벡터 술어 둘 이상) 조건의 결합 효과 — 본 연구는 단일 벡터 술어 한정. (3) Milvus·Qdrant·VBASE·DuckDB-vss 다른 엔진 호환성 — pgvector 외 4 엔진 매트릭스 검증.

Q10-2. History-aware Adaptive Sampling 우선순위는 어디인가요?

본 연구가 sample budget 안에서 표본 선택을 바꿔서 추정 정확도를 89% 개선했다면, 다음은 sample budget 자체를 줄이는 방향입니다. embedding 유사도로 과거 query를 clustering하고, 비슷한 query 그룹의 추정값을 신뢰구간과 함께 캐싱해요. 신뢰구간이 좁으면 sampling 자체를 skip — sample budget을 0에 가깝게 만드는 게 목표입니다.

Q10-3. inverse-variance weighting 같은 가중 결합 검토할 계획이 있나요?

네, 후속 검토 항목입니다. 분포 인지 method의 분산이 baseline보다 작아지는 조건 (예: 특정 paradigm·특정 cell) 을 찾아서, 가중 평균에서 분포 인지에 더 큰 가중치를 주면 negative finding을 positive로 뒤집을 가능성이 있어요. 정확한 가중치 추정 자체가 추가 비용이라 trade-off 검증이 필요합니다.

§11. 산업 적용

Q11-1. 실제 산업 환경에 바로 적용 가능한가요?

pgvector 패치는 비교적 직접 적용 가능합니다. 약 30줄 vector.c 패치이고, GUC 변수로 카디널리티를 주입하는 ExecutorRun hook이라 안전하게 토클 가능해요. 다만 학술 결론이 sel=0.001 평면에 한정되어 있어 실무 도입 전에 운영 환경 워크로드로 추가 검증이 필요합니다.

Q11-2. 패치 적용 비용은 어느 정도인가요?

코드 변경 약 30줄, 빌드·테스트 약 30분, 운영 적용 (postgres restart) 약 1분이에요. GUC `vector.injected_card` 환경 변수로 카디널리티를 주입하니까 기능 토클 가능하고, 기존 pgvector 사용자도 점진적 도입 가능합니다.

Q11-3. HTAP 환경 (혼합 워크로드) 에서 효과가 유지될까요?

정확히 검증한 적은 없지만, 본 연구의 결합 method가 OS 상태 변화·workload 변화에 더 robust한 점이 HTAP 환경에 유리하게 작용할 가능성이 큼니다. 그게 future work §G-2 산업 적용 framework의 핵심 검증 항목입니다.

Q11-4. pgvector 외 다른 벡터 DB는요?

PostgreSQL 계열 (DuckDB·vss·VBASE·CockroachDB pgvector 호환층) 은 plan-based optimizer라 비교적 직접 적용 가능. Native 벡터 DB (Milvus·Qdrant·Weaviate) 는 plan-based optimizer가 없거나 다르기 때문에 효과가 다르게 나올 거예요. 6/11 보고서에서 4 엔진 매트릭스로 통합 검증 예정입니다.

§12. cheat sheet — 외출 핵심 수치 + 발화 톤 룰

핵심 수치 (외우는 것)

수치	의미	발화
89.1% (1,344/1,508)	결합 vs baseline Q-error 우위 cell	"1,508 셀 중 1,344, 89.1%"
-4.38% (중앙값) · 3.86% (평균)	Q-error 개선 폭	"중앙값 4.4% 감소"
35.2% (531/1,508)	단독 대체 baseline 이긴 비율	"단독은 35%, 동전 던지기"
5.81x (977.6 vs 5,677ms)	baseline vs pgvector 가속	"5.81배 빠름"
+0.6% (5.9ms)	결합 vs baseline latency 격차	"결합이 5.9ms 더 느림, 사실상 동등"
5.77x (983.5 vs 5,677ms)	결합 vs pgvector 가속	"5.77배 빠름"
58.3% → 94.9% (91→148 / 156)	plan 회복률 baseline → 결합	"58에서 95%로 robust"
N=385	표본 예산 (Exqutor §V-B verbatim)	"385행"
trial 10·n_queries 1,000	paper §V-B 표준	"논문 표준"
chao_weighted offline 1위	P3 Streaming, 100% 우위	"스트리밍, 100%"
mhst2 엔진 plan 12/12	P6 양자화	"엔진 100%"
pca1d 균형 1위	offline + 엔진 양면 강	"균형 1위"
dual-Bernoulli ensemble	통제군 — 분포 인지 효과 분리	"베르누이 두 번 평균, 결합보다 더 정확"

발화 톤 룰 (조현빈 답변)

- 숫자부터 답 — "1,508 중 1,344, 89.1%" — 추측 없이 정본 수치
- 모르면 모른다고 — "그건 보고서 §4.X에 있는데 발표에서는 안 다뤘습니다"
- negative finding 정직 인정 — "분포 인지가 평균 효과 위에 추가로 더해주진 않습니다, honest limitation으로 명시했어요"
- 2~3문장 압축 — 더 길어지면 청자 잃음. 15~30초 안.
- ★ 결합이 0.6% 느림 정직 인정 — "솔직히 평균만 보면 0.6% 느립니다. 5.9ms는 noise 수준이고, 가치는 plan 안정성이예요."
- ★ method 명칭 정정 정직 — "5/24 적대 검증에서 발견한 부분인데, hilbert_real은 사실 PCA 2D + xy2d Hilbert hybrid예요."
- slide 9 P5 클러스터링 모순 정직 — "슬라이드 시각 단순화 때문이고, 실제 P1 Cluster 계열은 권장 제외했습니다."

답변 우선순위 (3분 안 핵심 — 시간 부족 시)

- 단일 개입 framing (Q2-3, 40초)
- 결합 89.1% 메커니즘 — 분포 인지 X, 평균 효과 ✓ (Q8-1·Q8-2, 40초)

3. chao 1위 / mhist2 엔진 1위 / pca1d 균형 1위 (Q5-1·Q5-3, 30초)
4. plan 94.9% vs latency 동등의 비대칭 구조 (Q6-1·Q6-2, 40초)
5. honest limitation 3종 (Q9-1, 30초)

부록 — 정보 참조

정보	경로
본 통합본 (이 문서)	submission/_drafts/속도는벡터_5_29_발표_QnA_통합본_20260528_182000.{md,pdf}
발표 슬라이드	submission/제출완료/속도는벡터_기말발표.pptx (13p)
발표 스크립트	submission/제출완료/기말발표_스크립트.pdf (13p)
6/11 보고서 신본	submission/_drafts/속도는벡터_6_11_최종보고서_20260523_215000.{md,pdf}
학부생 알고리즘 설명	submission/_drafts/속도는벡터_알고리즘_학부생수준_설명_paradigm_method별_20260527_001500.{md,pdf}
v13 측정 정보	_internal/cache/rq3/v13_summary.md
4-case raw 156 paired	submission/_drafts/archive/속도는벡터_plan회복_4case_raw_156paired_20260526_220000.csv
직전 carry (5/27 31질문)	submission/_drafts/속도는벡터_QnA_예상질문_답변_20260527_004000.{md,pdf}
직전 carry (5/28 14:55)	submission/_drafts/속도는벡터_5_29_발표_QnA_사전준비_20260528.md

작성: 2026-05-28 18:20 KST · 5/29(금) 기말발표 Q&A 통합본 · 발표자=강재현 · 답변=조현빈 담당 · 슬라이드 13p × 스크립트 13p × 카톡 5/27-5/28 정독 통합 · 12 카테고리 · 청중 시각 시뮬레이션 70+ 질문 · 구어체 답변 (15-30초)